

Ranting Cheatsheet

say! () placeholder syntax and the crate's public surface. Every output below is copy-pasted from a real call.

■ **case**, ^ ■ **article** ■ **plurality** ■ **case marker** ■ **verb / tense** ■ **degree** ■ **output** ■ **gotcha** ■ **note**

PLACEHOLDER ANATOMY

```
{[,^]?(article)?([+~]|[$#]?var)?(\\*['=@~%]|['=@~*%'])?noun(verb)(!word | !!word)}
```

, ^ case: force lower / upper, overriding the sentence-start default

noun a variable name or positional index 0, 1, ...

a an some the these those article

verb base verb, optionally tense-marked

+ - \$var #var plurality: force plural / singular / numeric-driven

!word !!word degree: comparative / superlative

` = @ ~ % \* ?** case marker: which grammatical form to render; or fused **\*=@\* `*~*%

Order is fixed; anything bracketed or parenthesized is optional.

ONE SENTENCE, ANNOTATED

```
say!("{=p can't} get in {`p} house.")  
→ Err("She can't get in her house.")
```

say! wraps the result in `Err(...)`; **ack!** gives `Ok(...)`
=p subject form of noun `p` → `She`, capitalized because it opens the sentence
can't verb inside the same placeholder, conjugated to agree with `p`
**`p** possessive determiner of the same noun → `her`

CASE MARKERS

Which grammatical form of the noun renders. Nouns:
Noun: `:new("Person", "he"), ("Jane", "I"), alex = ("Alex", "she")`.

(none) name · `{person walk}` → "Person walks"

= subject · `{=person walk}` → "He walks"

@ object · `{@0}` → "her"

**`** possessive determiner · `{`jane}` → "My"

~ possessive pronoun · `{~tarzan}` → "His"

% reflexive · "Only `{^%alex}` can decide that." → "Only Herself can decide that."

***** display name, verb still agrees · `{*tarzan who have}` book → "Tarzan who has book"

***= *@ *` \*~ \*%** fused: case-marks like the bare marker, but shows the **name** · `{the *=noun}` → article case-correct, name shown

? hidden — inflects but isn't shown · `{?=person are}` here, but `{=person}` stays invisible above. → "She is here, but she stays invisible above."

? composes: `{?the noun}` hides noun and article, still drives agreement.

GOTCHAS

No marker renders the name, not a pronoun. `{person walk}` → "Person walks". You almost always want `{=person walk}` → "He walks".

MACROS

say!(fmt, args...) Parses placeholders at compile time into a `format!()` call, returns `String`.

say_with!(&ctx, fmt, args...) Defers tense to `ctx.tense` at runtime; also threads `narration_person, register, dialect`.

ack!(fmt, args...) → `Ok(say!(...))`, a plain expression. Intended for "allow" responses.

nay!(fmt, args...) → `Err(say!(...))`, same shape.

heed!(template, input) Inverse direction: matches text against a template, returns captures.

ask!(speaker, audience, template, input) Parses input against template like `heed!()`, forwards the captures to audience's `Answerable::answer`. Returns `Option<String>`.

```
let r: Result<String, String> = ack!("{  
  {=p are} welcome.");  
→ Ok("She is welcome.")
```

The article adapts to plurality; the name does not. `{a dogs}` → "Some dog". Add `+` to pluralize the name too. **Never pass an already-conjugated verb.** `{=person goes}` → "He goess". Write the base verb and let the

VERB TENSES

Write the **base** verb; the marker conjugates it.

(none) present · `{=person walk}` → "He walks"

< past · `{=person <walk}` → "He walked" (irregular: "He went")

= present continuous · `{=person =walk}` → "He is walking"

> future · `{=person >walk}` → "He will walk"

<= past continuous · `{=person <=walk}` → "He was walking"

% present perfect · `{=person %walk}` → "He has walked" (irregular: "He has gone")

<% past perfect · `{=person <%walk}` → "He had walked" (irregular: "He had gone")

`say_with!(ctx, ...)` resolves these markers against `ctx.tense` at runtime instead; `say!()` is unaffected.

tense marker conjugate.

Noun: :new panics on an invalid subject. Use `try_new` when the subject isn't a literal.

CASE FORCING , AND ^

Only needed mid-sentence — a placeholder at sentence start auto-capitalizes.

The person is actually a `{, +0}`. irregular plural, forced lowercase → "The person is actually a people."

Only `{^%alex}` can decide that. forced uppercase mid-sentence → "Only Herself can decide that."

ARTICLES

a/an/some adapt to plurality; the is invariant; these/those become this/that when singular.

`{a dog} {the dog}` dog / "it" → "A dog" · "The dog"

`{a dogs} {the dogs}` dog / "they" → "Some dog" · "The dog"

`{these dog} {those dog}` singular → "This dog" · "That dog"

`{these 0} {those 0}` one / "they" → "These one" · "Those one"

Two adjacent heed! captures is a compile error — ambiguous with no literal text between them. **ack!/nay! are expressions, not returns.** Write `return ack!()` yourself if you want an early return.

NOUN

```
Noun::new(name, subject) -> Noun
// panics on an invalid subject
Noun::try_new(name, subject)
  -> Result<Noun, InvalidSubjectError>
```

Valid subjects: "I" "you" "thou" "he" "she" "it" "we" "ye" "they". Prefer try_new for non-literal subjects.

COMPARATIVE / SUPERLATIVE

```
say!("{?w !good} than that.", w)
  -> "better than that."
say!("{?w !!good} in class", w)
  -> "best in class"
```

Irregular table plus regular -er/-est (CVC doubling, y→i) and periphrastic more/most for longer adjectives. Resolved entirely at compile time — no context or subject agreement needed.

PLURALITY FORCING

```
{+0} book / "it" → "Books"
{-=0 do} plural-declared person → "It does"
{+=0 do} same person → "They do"
I see ${count apple}. count = 3 → "I see 3 apples."
{#count noun} count spelled out → "I see three apples."
I count {#count ?noun}. noun hidden → "I count three."
${$one noun} count = 1 → "I see 1 apple."
```

THE RANTING TRAIT

Required methods — every implementor defines all five. Ranting: Display, so any placeholder subject also works with plain {}.

name(&self, uc: bool) -> String struct name, #[ranting(name = "...")], or self.name when name = "\$"

subjective(&self) -> &str subject pronoun; "it" by default

is_plural(&self) -> bool usually derived from subjective(); override for you

inflect(&self, to_plural: bool, uc: bool) -> String singular/plural name form; uses singular_end/plural_end

skip_article(&self) -> bool omit the article — proper nouns, uncountables

Customization hooks — all default to None ("use built-in English rules"). Every call site invokes the _with_context variant (say!() passes None, say_with!() passes Some(ctx)), so overriding only that form is enough.

inflect_verb_custom verb conjugation — tense, plurality, person

inflect_pronoun_custom pronoun form, keyed by PronounCase: Subjective, Objective, PossessiveDeterminer, PossessivePronoun, Reflexive

inflect_article_custom article form — a/an/the/some, demonstratives
ctx: Option<&NarrationContext> is always a parameter, never read from self: an entity's subject is entity-owned, while tense/viewpoint/register/dialect are story-wide. This is how dialect forks are built without forking the crate.

NARRATIONCONTEXT

```
NarrationContext::new()
  .tense(Tense::Past)
  .narration_person(Person::Third)
  .register(Register::Formal)
  .dialect("pirate")
```

All fields are Option, builders chain, and the struct is Copy — reusable across calls.

Tense Present Past Future
PresentContinuous PastContinuous
PresentPerfect PastPerfect interpreted:
resolves the tense markers at runtime

Person First Second Third interpreted, but scoped: only overrides nouns declared "I"/"we". Third person always falls back to singular "they". we → Second renders "you" — one-way, not round-trippable.

Register Formal Neutral Casual inert unless an impl reads ctx.register in a *_with_context hook. None = no override; Some(Neutral) is a distinct explicit value.

dialect&'static str inert — fork-defined, same hook mechanism.

DERIVING RANTING

```
#[derive_ranting]
#[ranting(subject = "he", name = "Merlin")]
struct Wizard {}
```

Core attributes — grammatical function:

subject default "it"; "\$" reads a subject: String field

name default struct/enum name; "\$" reads a name: String field

singular_end default "" — suffix stripped when singularizing

plural_end default "s" — suffix added when pluralizing

Cosmetic attributes — display only:

plural_you false — whether subject = "you" is plural

uc false — name always starts uppercase

no_article false — skip articles (proper nouns, meals, uncountables)

WRAPPER TYPES

Box<T: Ranting> Delegates every Ranting method straight through to *self.

Many<T> wraps Vec<T> Collective noun phrase, renders "a, b and c". Plural whenever len() != 1 — zero included. Delegates through with exactly one item; built-in English for 0 or 2+. skip_article() is true when empty.

Maybe<T> wraps Option<T> Some(x) behaves exactly like x; None renders as nothing, singular, subject "it", article skipped.
They compose: Many<Box<Noun>>, Box<Many<Noun>>. Vec/Option can't implement Ranting directly — orphan rules.

HEED!() / ASK!() CAPTURES

heed!("take {item}", "take sword")
→ Some("sword")

heed!("give {item} to {target}", "give sword to guard") → Some(("sword", "guard"))

heed!("take {item}", "drop sword")
→ None

{name} one token
{name...} greedy, to the next literal or end of input
{ \$name } digits, parsed as u64
A bare value for 0–1 captures, a tuple for 2+, positional like say!(). No article, verb or pronoun markers here.

ask!(player, Dog, "pet dog", "pet dog") → Some("Woof!") · "kick dog" → None

Same template grammar, but captures go to the audience's Answerable: answer(&self, speaker: &dyn Ranting, captures: Self::Captures) -> String.

ADVANCED & RARELY USED

boxed_ranting_trait!(T) / ref_ranting_trait!(T) Generate Ranting impls for Box<dyn T> / &dyn T so trait objects work as placeholder subjects.

HeedMatcher(#[doc(hidden)]) backs every heed!() call site — one static matcher per site, regex compiled once via OnceLock. Not meant to be constructed directly.

PronounCase, InvalidSubjectError small supporting types — see their rustdoc.

FEATURE FLAGS & SCOPE

inclusive-pronouns on by default — docs/examples for gender-neutral singular "they".
debug — prints each say!() pattern with its generated format!() conversion at compile time.
Only ranting and ranting_derive are semver surface. ranting_core is internal and unversioned, even where re-exported.